

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: CE-1103 Algoritmos y Estructuras de Datos I.

Atributo Académico: Diseño (DI) para proyecto programado #1: Cyber
Defense Duel

Profesor: MSc. Andrés Vargas

Integrantes:

Ezequiel Bonilla Vega: 2025098474

Manuel Antonio Rojas L: 2020072613

Luis Fernando Ortiz M: 2022206801

18 de abril de 2026

I semestre

Parte 1

1. ¿Cuáles son las principales necesidades del sistema que desarrollaron (cliente-servidor, interacción en tiempo real, persistencia, etc.)?

Se considera la interacción entre el cliente y el servidor como una de las principales necesidades desarrolladas para el sistema, debido a que la mayor parte del programa depende de la comunicación entre ambos.

El tiempo de interacción es rápido. El servidor no necesita realizar muchas operaciones con múltiples clientes, ya que solamente se trata de dos clientes y mensajes sencillos de actualización de datos.

La persistencia se logra por medio del servidor, este tiene el control sobre la “base de datos” JSON. El sistema de persistencia si es importante, especialmente para el funcionamiento de la validación de datos del usuario en el momento de login.

2. ¿Qué requerimientos funcionales consideran críticos para que el sistema cumpla su objetivo (ej: login, comunicación entre clientes, sincronización de estado, etc.)?

Como se mencionó anteriormente, el sistema más importante o crítico es la comunicación entre el servidor y el cliente. Por lo tanto, la conexión de la clase cliente con el resto de las clases que conforman el programa del lado del usuario fue vital.

3. ¿Qué requerimientos no funcionales identificaron en el proyecto (ej: rendimiento, escalabilidad, usabilidad, mantenibilidad)?

Se implementó esta arquitectura, porque es la estándar para comunicar múltiples computadoras en una red. El cliente se diseñó con el fin de funcionar como el punto de partida de datos y recepción de instrucciones por parte del servidor. Este se comunica directamente con el mediador de las clases que conforman el programa del usuario.

El servidor se encarga de recibir mensajes de cada cliente y actualizar los datos de estos. También maneja la parte de datos permanentes. Esto lo hace funcionar como un hub central de información y comunicación.

En el apartado de la lógica del juego se partió de un diseño modular, en este se creó un objeto general con configuraciones básicas de hitbox y sprites, esto facilitó la creación del resto de clases como el jugador, enemigos y las balas ya que se podían agregar parámetros que los 3 implementaban de manera muy sencilla, solo modificando la clase GameObject.

Posteriormente, tenemos también la clase GameManager en la cuál las instancias de todos estos objetos se consolidan, además de agregar las condiciones cuando interactúan entre sí. Este tipo de diseño permite que el game designer (En un entorno real de desarrollo de juegos) pueda experimentar dentro del ciclo jugable con objetos previamente configurados.

Finalmente, la parte de interfaz de usuario solo tiene que generar un nuevo manager, pasarle la scene y mostrarla, toda la lógica del juego está aparte de la parte gráfica de interfaces lo cuál permite una versatilidad enorme. En especial a la hora de debugear al poder llamar el juego de manera independiente.

En términos de rendimiento se tomaron en cuenta varios errores de manejo de objetos especialmente con las balas, ya que al saber que la memoria es limitada es totalmente innecesario y hasta peligroso instanciar nuevos objetos cada vez que se hace llamada a funciones como shoot(); Para este problema, se crearon stacks y listas de objetos de la clase Bullet, al ya estar instanciadas el programa solo tiene que traerlas a pantalla y sacarlas una vez estén fuera de la vista del jugador, esto da la ilusión de que las balas se crean y no que se están reutilizando, evitando un posible overflow de memoria, dentro del programa están instanciadas una cantidad límite de balas suficientes que aguanten una ejecución normal del juego.

4. ¿Cómo consideraron la seguridad dentro del sistema? (ej: validación de usuarios, integridad de datos, manejo de errores en sockets)

El archivo JSON que contiene las credenciales de los usuarios se encuentra dentro del repositorio sin encriptar lo cuál podría llevar a que cualquier usuario pueda modificar los datos, en caso de que sea borrado el archivo se vuelve a crear por lo qué específicamente no sería un problema.

Por otra parte el sistema comprueba que si el usuario existe y si la contraseña es correcta tanto para login como para registrarse, en el primer caso evita que se ingrese con un usuario inexistente o que se ingrese con un usuario que no le pertenece, para el registro, evita que existan dos usuarios con el mismo nombre y con ello errores del sistema.

5. ¿Qué limitaciones técnicas del proyecto influyeron en su diseño? (ej: no uso de estructuras de Java, uso de JSON, sockets, etc.)

Para este proyecto realmente no se encontraron tantas limitaciones con los recursos utilizados, para este alcance local el intercambio de datos en JSON sin tipado fuerte, favoreció la flexibilidad pero con mayor riesgo de errores por claves mal escritas, esta fue una implementación fácil y acertada, así como el uso de sockets TCP en bajo nivel, que obligó a crear un protocolo propio de mensajes y manejo manual de conexiones/hilos en una arquitectura orientada a partida 1v1 (dos clientes), que reduce complejidad inicial pero limita expansión a más jugadores o salas múltiples.

Podría caber la posibilidad de que las estructuras de datos (listas, pilas, colas) en lugar de java.util, lo que simplificó el enfoque académico pero aumentó la complejidad de implementación, esto podría llevar a limitantes pero eso depende totalmente de cómo se haya implementado dichas estructuras, para este proyecto no fue el caso.

Finalmente, el mayor limitante encontrado es en sí la librería que se usó para programarlo que definitivamente no está diseñada para manejar una aplicación como un juego; ya que esta se destaca por su uso para creación de interfaces gráficas.

6. ¿Cómo influye el uso de recursos del sistema (CPU, memoria, red) en su diseño?

El programa debería ejecutarse en equipos hasta de bajos recursos, ya que solo se necesita tener la JVM (Java Virtual Machine) para ejecutarse. Con el juego se construyeron estructuras de datos especiales para solapar posibles problemas de overflow de memoria, como se explicó previamente con las balas, por lo cuál una vez ejecutado el programa este no debería tomar más memoria de la que ya está definida.

Finalmente, todas las conexiones son locales y para mayor facilidad a la hora de su ejecución se decidió usar un switch que permita una conexión estable entre los equipos conectados.

7. ¿Qué impacto tendría su sistema si fuera utilizado en un entorno real con múltiples usuarios?

En todo caso la posible ejecución del juego sería en un entorno local de red, esto haría que las partidas dependan totalmente de la cantidad de equipos que se encuentren conectados a la misma red, aunque estén jugando el juego o no. Una enorme cantidad de equipos simultáneamente podría ralentizar un poco la detección del usuario y la actualización de los datos del oponente más el juego no.

La ventaja del diseño que implementamos es su capacidad de correr el juego sin la necesidad enteramente de una conexión a internet, aunque en la clase GameManager existen las llamadas al servidor, estas se ejecutan de manera asíncronas, no alterando el juego de ninguna manera, en el peor de los casos, el juego se ejecutaría sin que los datos del rival se estén actualizando.

Parte 2

1. ¿Qué alternativas de arquitectura consideraron para el sistema? (ej: cliente-servidor vs local, síncrono vs asíncrono)

La arquitectura cliente-servidor fue la adecuada a implementar debido a que la idea del proyecto era que dos jugadores jugaran simultáneamente en cada uno de sus equipos, una implementación local habría extirpado toda la

parte del servidor y hubiese creado nuevos retos con la lógica y el input. En todo caso se tendrían que implementar dos set de teclas para cada uno de los jugadores y crear dos instancias del gamemanager para ejecutar el juego en dos ventanas, cada una perteneciendo a un jugador.

Por otra parte, la validación de usuario se hubiese visto afectada creando solamente usuarios locales de cada computadora, además de la pregunta ¿Como válidas a dos usuarios diferentes en la misma computadora?

La implementación de cliente-servidor también deja la puerta abierta para una implementación más compleja a futuro, donde en vez de conectarse a una red local, se podrían conectar diferentes jugadores a un servidor remoto, creando un gameplay donde ambos jugadores no tienen que estar conectados a la misma red de internet, además es bastante fácil de implementar como tenemos montado el proyecto.

El juego se ejecuta de manera asíncrona, como se comentó en la parte 1, esto permite que las conexiones no ralenticen o detengan el juego, ejecutándose este de manera fluida todo el tiempo y de manera simultánea con el servidor para la actualización de datos.

2. ¿Por qué eligieron la arquitectura cliente-servidor para este proyecto?

Aparte de ser uno de los requerimientos de proyecto, esta era la única implementación que permitía tener a dos clientes conectados simultáneamente ejecutando el programa

3. ¿Qué alternativas consideraron para la implementación de estructuras de datos y por qué eligieron las utilizadas?

Dentro del proyecto se decidieron crear la mayoría de estructuras de datos vistos en clase aunque no se utilizaron dentro del proyecto, las dos más utilizadas fueron el stack y las listas, las primeras para instanciar las balas que los enemigos y el jugador usan, usando el principio LIFO nos permite sacar balas para su uso y volver a insertarlas una vez hayan sido utilizadas, estas listas se utilizan para trabajar con las balas que se encuentran

en pantalla, esto permite recorrer la lista y definir el comportamiento de la bala específica.

4. ¿Qué decisiones tomaron para organizar las clases del sistema? (ej: separación de responsabilidades)

Alrededor del diseño del juego, el game loop se decidió dividir cada entidad en un objeto diferente basándose de un padre con parámetros que compartían todas las entidades del juego. Además de delegar toda la lógica del juego al manager, esto facilita su implementación, el cliente no tiene que saber qué o cómo funciona el gameManager solo tiene que instanciarlo y se ejecutará el juego.

5. ¿Evaluaron diferentes formas de manejar la comunicación entre cliente y servidor? ¿Cuáles y por qué eligieron la actual?

En sí, la estructura de la comunicación entre cliente y servidor estaba bien definida desde un inicio, esto puede apreciarse desde el diagrama de clases que sentó la idea de cómo se iba a construir.

6. ¿Qué ventajas y desventajas tiene su diseño actual?

De la parte del game loop dentro del gameManager se puede pulir el sistema para dividir la lógica del juego y como los elementos interactúan entre sí y el diseño de niveles, en donde fácilmente pueda existir hasta un editor de niveles gráfico que permita a cualquiera utilizar los elementos y posicionarse fácilmente sin preocuparse por su lógica.

Incluso se podría haber mejorado la abstracción alrededor de cómo interactúan los elementos entre sí, separando cada una de estas en funciones más pequeñas, como el siguiente ejemplo proporcionado por la IA de Google Gemini

```
// Pseudocódigo del bucle principal  
  
while (appRunning) {  
  
    if (editorMode) {
```

```

        processEditorInput();

        updateEditorUI();

        renderEditorGizmos();

    } else {

        processGameInput();

        updateGamePhysics();

        renderGameWorld();

    }

    renderGUI(); // La UI del editor siempre se renderiza encima
}

```

7. Si tuvieran que rediseñar el sistema, ¿qué mejorarían y por qué?

Como se mencionó anteriormente la parte de lógica del juego podría mejorar, dividiendo aún más su diseño en diferentes funciones para tener un game loop más ordenado.

Se agregaría un apartado especial para borrar cuentas o al menos cambiar la contraseña de cuentas existentes, para evitar cuentas sin uso inaccesibles por el desconocimiento de las credenciales.

Por último, se intentarían agregar más escenarios o niveles que contengan un diseño diferente y que ofrezcan nuevos retos a los jugadores para que puedan seguir usando el programa y no sea tan repetitivo.

8. ¿Cómo su solución cumple con los requerimientos identificados en la Parte 1?

Aún y con algunos fallos que el sistema pueda tener, el sistema cumple con todos los requerimientos del proyecto. Cumple satisfactoriamente la parte de conexiones provisionando un sistema funcional de logueo y registro de usuarios, esto por medio de la implementación de sockets y una

clase mediadora que manejaba las conexiones; además de la lectura y escritura de archivos json.

El usuario tiene un marco limitado de selección con el avatar y el mapa que se refleja en el gameplay aunque sea con solo cambios estéticos. También las interfaces están creadas de manera amigable al usuario tratando de guiar al mismo visualmente, todo esto se logró gracias a la librería Java FX que posee un amplio marco de widgets y aspectos que permiten crear una aplicación agradable.

También gracias a la conexión asíncrona, el jugador puede estar en su partida aunque la conexión falle o se ralentice, además siempre se muestra el ganador de la partida una vez haya estado finalizado. Esto se logró al separar ambos apartados (Servidor-cliente y el juego) en módulos separados que se ejecutan al mismo tiempo pero que no dependen uno del otro excepto cuando se inicia el juego o uno de los dos jugadores muere.

El juego como tal y su loop, se logró implementar usando JavaFX que pese no ser específicamente para crear videojuegos, muchas de sus clases se lograron implementar para simular un motor, por ejemplo el uso de Rectangle como coliders, Image como sprites y los diferentes métodos de posiciones para el movimiento.